

RTL Design Challenge Specification

Streamed Intermediate-Result Processing Module

1. Introduction

1.1 Background

Modern computing systems usually divide large computational tasks into multiple processing stages. Instead of producing the final output immediately, an external computation engine generates intermediate results over multiple cycles. These intermediate results must be managed carefully before they become available as final outputs.

To achieve continuous operation without interrupting the computation engine, a dedicated hardware module is required to receive incoming partial results, process them according to a predefined flow, and expose completed outputs through a dedicated interface.

The internal implementation of this module is left entirely to the designer. Any architecture that satisfies all functional and timing requirements described in this specification is considered valid.

1.2 Objective

The objective of this challenge is to design a synthesizable RTL hardware module that manages streamed intermediate computation results.

The designed module shall:

- Receive incoming partial computation results.
- Preserve intermediate data whenever necessary.
- Update stored values according to the required processing flow.
- Produce completed output words.
- Support continuous streaming operation.
- Operate correctly under all timing conditions described in this document.

1.3 Scope

Only the module described in this specification shall be implemented.

The external computation engine is **not** part of this challenge.

The external consumer that reads completed outputs is also **not** part of this challenge.

Both are assumed to behave according to the interface described later.

1.4 Design Goals

A successful implementation should satisfy the following objectives:

- Functional correctness.
- Clean RTL architecture.
- Synthesizable implementation.
- Efficient resource utilization.
- Parameterized design whenever applicable.
- Robust handling of all specified operating conditions.

2. System Overview

The designed module is positioned between two independent hardware blocks.

The first block continuously generates intermediate computation results.

The second block consumes completed output words.

The designed module acts as the interface between both sides.

Its responsibility is limited to:

- Receiving incoming partial data.
- Managing intermediate data.
- Updating stored values whenever required.
- Making completed outputs available.
- Maintaining correct operation throughout continuous execution.

The behavior of both external blocks is assumed to be correct and compliant with the interface timing described later.

3. Configuration Parameters

The module shall support parameterization to allow integration into systems with different data organizations.

Unless otherwise specified, all parameters are compile-time constants.

Parameter	Value	Description
DATA_WIDTH	16	Width, in bits, of a single data element.
PARTIAL_SIZE	4	Number of data elements contained in one Partial Transaction.
OUTPUT_SIZE	8	Number of data elements contained in one Output Word.
ADDR_WIDTH	8	Width of logical output-address values.
MAX_SEGMENTS_PER_RANGE	100	Maximum number of logical processing segments permitted within a single Processing Range. Used in Chapter 7 to size buffering and latency.

Constraint: OUTPUT_SIZE shall be chosen such that $\text{OUTPUTS_PER_SEGMENT} = \text{PARTIAL_SIZE}^2 / \text{OUTPUT_SIZE}$ (Section 3.1) is an integer. With the default values above, $\text{OUTPUTS_PER_SEGMENT} = 16 / 8 = 2$.

3.1 Data Element Mapping

This section fixes the numerical relationship between Partial Transactions, logical processing segments, and Output Words. This relationship follows directly from the parameters above and applies regardless of the internal architecture chosen by the designer.

PARTIALS_PER_OUTPUT = $\text{OUTPUT_SIZE} / \text{PARTIAL_SIZE}$ the number of consecutively accepted Partial Transactions whose data elements, concatenated in acceptance order, exactly fill one Output Word.

OUTPUTS_PER_SEGMENT = $\text{PARTIAL_SIZE} / \text{PARTIALS_PER_OUTPUT}$ the number of complete Output Words produced by every logical processing segment (a segment is defined in Section 5.2 as PARTIAL_SIZE consecutively accepted Partial Transactions).

With the default parameter values ($\text{PARTIAL_SIZE} = 4$, $\text{OUTPUT_SIZE} = 8$):

- **PARTIALS_PER_OUTPUT** = 2 so every 2 consecutively accepted Partial Transactions supply exactly one Output Word's worth of data.
- **OUTPUTS_PER_SEGMENT** = 2 so every logical processing segment (4 Partial Transactions) produces exactly 2 Output Words.

A worked numeric example using these values is provided in Section 6.6.

3.2 Partial Transaction

The external computation engine transfers data using **Partial Transactions**.

Each Partial Transaction consists of exactly PARTIAL_SIZE data elements.

Example: with $\text{PARTIAL_SIZE} = 4$ and $\text{DATA_WIDTH} = 16$, every clock cycle in which **partial_valid** is asserted, the module captures 4 elements on **partial_data** ($4 \times 16 = 64$ bits total), each element represented in 16 bits.

A Partial Transaction is transferred only when **partial_valid** is asserted.

Whenever **partial_valid** is high, the receiving module shall capture the complete transaction during the same clock cycle.

3.3 Output Word

The module exposes processed data through **Output Words**.

Each Output Word contains exactly OUTPUT_SIZE data elements.

An Output Word shall become visible only after all required processing for that output has completed.

Once available, an Output Word shall remain stable until it is no longer being requested by the downstream interface (see Section 5.4).

3.4 Streaming Model

The module is designed for continuous streaming operation.

Incoming Partial Transactions may continue arriving while previously completed Output Words are simultaneously available to the downstream interface.

The implementation shall therefore support concurrent **input reception, intermediate data management, and output delivery** whenever protocol timing permits.

3.5 Module Interface

The following table summarizes the external interface of the module. All signals are synchronous to clk unless otherwise specified.

Signal	Dir.	Width	Description
clk	In	1	System clock.
rst_n	In	1	Asynchronous, active-low reset.
partial_valid	In	1	Indicates a valid Partial Transaction is present on partial_data.
partial_data	In	PARTIAL_SIZE × DATA_WIDTH	Incoming Partial Transaction.
segment_step	In	1	Indicates that processing shall advance to the next logical processing segment within the current processing range, according to the active traversal direction.
phase_change	In	1	Indicates a transition between consecutive processing phases within the current processing range.
next_range	In	1	Indicates that processing of the current processing range has completed and that subsequent Partial Transactions belong to a new processing range.
operation_done	In	1	Indicates that no additional Partial Transactions belong to the current processing operation.
output_addr	In	ADDR_WIDTH	Logical address of the Output Word requested by the downstream interface.

Signal	Dir.	Width	Description
output_valid	Out	1	Asserted when the Output Word at the requested output_addr is complete and available on output_data.
output_data	Out	OUTPUT_SIZE × DATA_WIDTH	Completed Output Word corresponding to the requested output_addr.

The functional behavior of these signals is defined in the following chapters.

4. Interface Protocol

This chapter defines the communication protocol between the module and its external interfaces.

Only the externally observable behavior is specified. The internal implementation remains entirely implementation-dependent.

4.1 Clock and Reset

The module shall operate synchronously using the rising edge of clk.

The **rst_n** signal is an active-low asynchronous reset.

Whenever **rst_n** is asserted, the module shall immediately enter its reset state regardless of the clock.

During reset:

- output_valid shall remain deasserted.
- No Output Word shall be considered valid.
- Any intermediate processing state shall be discarded.

Following the deassertion of **rst_n**, the module shall be ready to accept new Partial Transactions beginning with the next rising edge of clk.

4.2 Signal Validity Rules

The following rules apply to all external interfaces.

Input Data

A Partial Transaction shall be considered valid only while **partial_valid** is asserted.

Whenever **partial_valid** is asserted, **the value present on partial_data** shall be sampled during the same clock cycle.

No assumptions shall be made regarding the value of **partial_data** when **partial_valid** is deasserted.

Control Signals

All control signals are generated by the external computation engine.

The designed module shall use **these signals only** as control inputs. The module shall neither generate nor modify any control signal.

Control signals may be asserted independently or simultaneously **within the same clock cycle**.

Whenever multiple control signals are asserted simultaneously, the module shall correctly process all asserted signals during that clock cycle, following the hierarchy defined in Section 5.3.

Output Signals **output_valid** indicates that the value on **output_data** is the completed Output

Word for the address

currently presented on **output_addr**.

The value of **output_data** shall be considered valid only while **output_valid** is asserted.

No assumptions shall be made regarding **output_data** when **output_valid** is deasserted.

4.3 Streaming Rules

The module shall support continuous streaming operation.

The external computation engine is expected to transmit Partial Transactions as a continuous stream whenever data is available.

During normal operation, **partial_valid** may remain asserted across multiple consecutive clock cycles, allowing a continuous sequence of Partial Transactions to be transferred without idle cycles between them.

The module shall be capable of accepting one Partial Transaction during every clock cycle while **partial_valid** remains asserted.

The deassertion of **partial_valid** indicates only that no valid Partial Transaction is present during that clock cycle. It does not, by itself, indicate completion or termination of the current processing operation.

Incoming Partial Transactions may continue arriving while completed Output Words are simultaneously presented on the output interface.

The availability of completed Output Words shall not prevent the acceptance of new Partial Transactions, and continuous reception of Partial Transactions shall not prevent completed Output Words from being presented whenever they become available.

The module shall correctly support concurrent input reception, internal processing, and output generation whenever permitted by the interface protocol.

5. Processing Behavior

This chapter defines the functional behavior of the module during normal operation: how incoming Partial Transactions are accepted, how control signals influence the processing flow, how temporary interruptions are handled, and under which conditions Output Words are produced.

The descriptions in this chapter define only the externally observable behavior of the module. The internal implementation used to achieve this behavior is left entirely to the designer.

5.1 Processing Lifecycle

A processing operation begins when the module accepts the first valid Partial Transaction of a new processing operation. A Partial Transaction shall be accepted only during a clock cycle in which `partial_valid` is asserted.

Once the first Partial Transaction has been accepted, the current processing operation shall remain valid until it is completed or the module is reset.

Processing shall advance only during clock cycles in which `partial_valid` is asserted. The deassertion of `partial_valid` indicates only that no valid Partial Transaction is available during the current clock cycle; it shall not be interpreted as completion or termination of the current processing operation.

Whenever `partial_valid` is deasserted, the module shall suspend the current processing operation while preserving its complete processing context. Temporary interruptions of the input stream shall occur only at valid processing boundaries generated by the external computation engine; the module shall therefore never be required to suspend processing within an incomplete logical processing segment.

During any temporary interruption, no processing context associated with the current processing operation shall be discarded, modified, overwritten, reinitialized, or otherwise invalidated solely because `partial_valid` is deasserted.

When `partial_valid` is asserted again, the module shall resume processing from exactly the point at which it was suspended. (The processing state after resumption shall be functionally identical to the state that would have existed as the interruption had never occurred).

The assertion of `operation_done` indicates that the Partial Transaction accepted during the current clock cycle is the final Partial Transaction of the current processing operation. No subsequent Partial Transaction shall belong to the current processing operation.

Completion of a processing operation shall not be assumed immediately upon the assertion of `operation_done`. The module may continue any remaining processing required to complete the current processing operation and make the corresponding Output Words available; no new Partial Transactions will arrive for this operation.

The assertion of the asynchronous reset signal shall immediately terminate the current processing operation. Any processing context associated with the interrupted operation shall be discarded. The first valid Partial Transaction accepted after reset shall always belong to a new processing operation.

5.2 Processing Modes

Each processing range shall be processed through two consecutive processing modes: **initial_partials** and **accumulated_partials**.

During **initial_partials**, each accepted Partial Transaction shall generate the initial value of the corresponding Output Word within the current processing range.

Upon the designated processing boundary generated by the external computation engine (**phase_change**), the module shall transition to **accumulated_partials** while remaining within the same processing range.

During **accumulated_partials**, newly accepted Partial Transactions shall continue contributing to the Output Words belonging to the current processing range. Instead of generating Output Words for a new range, each contribution shall be accumulated with the corresponding Output Word previously produced, and the updated result shall replace the previous value at the same output location.

The module shall remain in **accumulated_partials** until the external computation engine indicates that processing of the current processing range has been completed. The module shall then begin processing the next processing range using **initial_partials**, after which the same sequence repeats.

Exact data operation (normative). To ensure every implementation produces identical output values for the same input stream, the following two operations are fixed and shall be implemented exactly as stated:

Generate initial value for each group of PARTIALS_PER_OUTPUT consecutively accepted Partial Transactions (Section 3.1), the OUTPUT_SIZE elements of the resulting Output Word are formed by concatenating those Partial Transactions in acceptance order (the first accepted transaction supplies elements [0 : PARTIAL_SIZE-1], the next supplies elements [PARTIAL_SIZE : 2×PARTIAL_SIZE-1], and so on). This concatenated value is stored as-is.

Accumulate the next group of PARTIALS_PER_OUTPUT consecutively accepted Partial Transactions is concatenated the same way, then added element-wise (per DATA_WIDTH-bit element) to the value already stored for that Output Word. The sum replaces the previously stored value.

5.3 Control Signal Behavior

The module shall interpret the control signals according to the processing boundaries generated by the external computation engine. These control signals define the progression of the current processing operation and shall be evaluated only during clock cycles in which `partial_valid` is asserted.

The control signals follow a hierarchical relationship. The assertion of a higher-level control signal implies the simultaneous assertion of all lower-level control signals during the same clock cycle:

Hierarchical Order	Control Signal
Level 1 (lowest)	<code>partial_valid</code>
Level 2	<code>segment_step</code>
Level 3	<code>phase_change</code>
Level 4	<code>next_range</code>
Level 5 (highest)	<code>operation_done</code>

Every control signal defined in this section is associated with the Partial Transaction accepted during the same clock cycle. Accordingly, whenever a control signal is asserted, the corresponding Partial Transaction shall be accepted concurrently with the assertion of `partial_valid`. If multiple hierarchical control signals are asserted simultaneously, all such signals shall coincide with the acceptance of that same Partial Transaction.

Accordingly:

- The assertion of `segment_step` shall imply the assertion of `partial_valid`.
- The assertion of `phase_change` shall imply the simultaneous assertion of both `segment_step` and `partial_valid`.
- The assertion of `next_range` shall imply the simultaneous assertion of `phase_change`, `segment_step`, and `partial_valid`.
- The assertion of `operation_done` shall imply the simultaneous assertion of `next_range`, `phase_change`, `segment_step`, and `partial_valid`.

The reverse implication does not apply: the assertion of a lower-level control signal shall not imply the assertion of any higher-level control signal.

The intervals between successive assertions of the control signals are determined entirely by the external computation engine and may vary throughout a processing operation. The module shall not assume any fixed timing relationship or constant interval between successive assertions of any control signal.

5.3.1 Control Signal Association

Every Partial Transaction shall be transferred only during a clock cycle in which `partial_valid` is asserted. Within every logical processing segment, the first ($\text{PARTIAL_SIZE} - 1$) Partial Transactions shall be accompanied only by `partial_valid`. The final (PARTIAL_SIZE) Partial Transaction shall always be accompanied by `segment_step`. Whenever that final Partial Transaction also corresponds to a processing-phase boundary, a Processing Range boundary, or the completion of the processing operation, the corresponding higher-level control signals (`phase_change`, `next_range`, and `operation_done`, respectively) shall be asserted simultaneously with that same final Partial Transaction. Consequently, whenever multiple hierarchical control signals are asserted, they shall all coincide with the acceptance of the same final Partial Transaction of the logical processing segment and shall not indicate separate or subsequent Partial Transactions.

5.3.2 `partial_valid`

The assertion of `partial_valid` indicates that a valid Partial Transaction is present on `partial_data` during the current clock cycle.

`partial_valid` may remain asserted for any number of consecutive clock cycles, allowing continuous reception of Partial Transactions without idle cycles between successive transfers.

The deassertion of `partial_valid` indicates only that no valid Partial Transaction is available during the current clock cycle. **It shall not terminate** the current processing operation, **invalidate** any previously accepted Partial Transaction, or **modify** the current processing context.

Whenever `partial_valid` is asserted again, the module shall resume processing from the preserved processing context as specified in Section 5.1.

5.3.3 `segment_step`

The assertion of `segment_step` indicates that the Partial Transaction accepted during the same clock cycle is the final Partial Transaction associated with the current logical processing segment. Accordingly, `segment_step` shall always be asserted concurrently with the acceptance of that final Partial Transaction (PARTIAL_SIZE Partial Transactions, per Section 3.1).

Under continuous input streaming, with `partial_valid` asserted on every clock cycle, `segment_step` shall be asserted once for every PARTIAL_SIZE consecutively accepted Partial Transactions.

Subsequent Partial Transactions shall belong to the next logical processing segment within the current processing range.

The assertion of `segment_step` shall not terminate the current processing range or the current processing operation. The module shall continue processing all previously accepted Partial Transactions in accordance with the interface protocol.

The timing of `segment_step` assertions is determined entirely by the external computation engine.

5.3.4 phase_change

The assertion of phase_change indicates a processing boundary within the current processing range.

If the current processing range is being processed in initial_partials mode, the assertion of phase_change shall terminate that mode and initiate accumulated_partials mode. All subsequently accepted Partial Transactions shall continue contributing to the Output Words belonging to the same processing range.

If the current processing range is already being processed in accumulated_partials mode, the assertion of phase_change shall not initiate a new processing mode. Instead, it shall indicate a transition to the alternate traversal direction for all subsequently accepted Partial Transactions within the same processing range.

The assertion of phase_change shall not terminate the current processing range, initiate a new processing range, terminate the current processing operation, or invalidate any previously accepted Partial Transactions.

Each logical processing segment consists of PARTIAL_SIZE consecutively accepted Partial Transactions. During initial_partials mode, the segments within a processing range are processed sequentially from the first segment to the last.

Upon entering accumulated_partials mode, the accumulation process initially proceeds in the reverse segment order, beginning with the last segment of the current processing range and continuing toward the first segment.

Upon the next assertion of phase_change, the accumulation direction reverses, and segments are again processed from the first to the last. Each subsequent assertion of phase_change during accumulated_partials mode alternates the accumulation direction between forward and reverse while remaining within the same processing range.

If partial_valid remains continuously asserted, the behavior associated with the phase_change assertion shall take effect immediately after the associated Partial Transaction has been accepted.

The timing of phase_change assertions is determined entirely by the external computation engine.

5.3.5 next_range

The assertion of next_range indicates that processing of the current processing range has been completed and that the next accepted Partial Transaction shall belong to the next processing range.

Upon accepting the associated Partial Transaction, the processing results associated with the completed processing range shall be considered final. No subsequently accepted Partial Transaction shall contribute to any Output Word belonging to the completed processing range.

The module shall terminate processing of the completed processing range and shall immediately initiate processing of the next processing range.

Each newly initiated processing range shall always begin in initial_partials mode, regardless of the processing mode or traversal direction associated with the previously completed processing range.

Processing of the new range shall begin with the first logical processing segment and proceed sequentially toward the last, until the next phase_change assertion occurs.

Each subsequent assertion of next_range finalizes the currently active processing range and initiates processing of the next processing range in the same manner. This continues identically for each successive processing range until the processing operation is completed.

The assertion of next_range shall not terminate the current processing operation unless operation_done is asserted during the same clock cycle.

If partial_valid remains continuously asserted, processing of the next processing range shall begin immediately after the Partial Transaction associated with the next_range assertion has been accepted.

The timing of next_range assertions is determined entirely by the external computation engine.

5.3.6 operation_done

The assertion of operation_done indicates that the current processing operation has received its final Partial Transaction. During the clock cycle in which operation_done is asserted, the module shall not accept a further Partial Transaction after the one accepted on that same cycle.

Since operation_done is the highest-level control indication in the protocol hierarchy, its assertion shall coincide with the assertion of next_range, phase_change, segment_step, and partial_valid during the same clock cycle.

When multiple hierarchical control signals are asserted simultaneously, the behavior associated with the highest asserted hierarchy level shall take precedence. The lower-level assertions shall be interpreted only as protocol requirements and shall not initiate independent behavioral transitions.

Upon accepting the associated Partial Transaction, the current processing range shall be considered finalized, and the current processing operation shall enter its completion stage.

No subsequently accepted Partial Transactions shall belong to the completed processing operation.

The assertion of operation_done shall not imply that all finalized Output Words have already been generated. Output generation may continue until all finalized Output Words associated with the completed processing operation have been produced. The processing operation shall be considered fully complete only after all finalized Output Words associated with that operation have been generated.

If partial_valid remains continuously asserted, the Partial Transaction accepted concurrently with operation_done shall be processed according to the behavior defined above.

The timing of operation_done assertions is determined entirely by the external computation engine.

5.4 Output Interface Behavior

5.4.1 output_valid

The output_valid signal indicates that the Output Word currently requested on output_addr has completed processing and is present on output_data.

The module shall assert output_valid whenever the Output Word at the requested address has been finalized.

The output_valid signal shall be deasserted whenever the requested address does not correspond to a finalized Output Word (for example, because that Output Word has not yet completed processing, or output_addr does not reference a valid produced output).

5.4.2 output_addr output_addr is driven by the downstream interface and specifies the logical address of the Output Word currently being requested. Its direction is an input to the module, as shown in Section 3.5.

On every clock cycle, the module shall compare the requested address against the logical addresses of the Output Words it has finalized so far in the current processing operation. If the requested address matches a finalized Output Word, the module shall present that word on output_data and assert output_valid for as long as that address remains requested.

Address assignment the module shall assign logical addresses to Output Words sequentially, in the order they are finalized: the first Output Word finalized within a processing operation is assigned address 0, and each subsequently finalized Output Word is assigned the next incremental address value. This numbering restarts at 0 at the beginning of every new processing operation (i.e., after reset or once a new operation begins).

The value of output_addr shall not affect the generation or contents of any Output Word it only selects which already-finalized Output Word, if any, is exposed on output_data.

5.4.3 output_data

The output_data signal presents the finalized Output Word corresponding to the address currently requested on output_addr, as determined in Section 5.4.2.

Whenever output_valid is asserted, the value presented on output_data is valid for use by the downstream interface.

The module shall continue serving output_data for as long as the requested address corresponds to a finalized Output Word; if the requested address changes to one that is not yet finalized, output_valid shall be deasserted until that Output Word is complete.

6. Processing Sequence

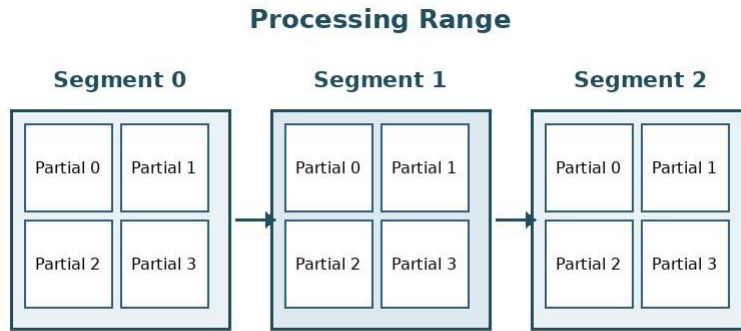


Figure 1. An Example of Processing Range with three logical processing segments (Segment 0–2). Each box represents one Partial Transaction; the 2×2 arrangement and the numbering shown inside each segment are illustrative layout only and do not indicate transmission order. The actual acceptance order within every segment is strictly sequential: Partial 0 → Partial 1 → Partial 2 → Partial 3, as described below.

6.1 Initial Processing Sequence

The figure illustrates a Processing Range consisting of three logical processing segments. Each logical processing segment contains four consecutive Partial Transactions, identified as Partial 0 through Partial 3.

At the beginning of every Processing Range, processing shall always start in `initial_partials` mode.

The first accepted Partial Transaction corresponds to Partial 0 of Segment 0. Subsequent Partial Transactions continue sequentially within the same logical processing segment until Partial 3 has been accepted.

Throughout the reception of all four Partial Transactions, `partial_valid` shall remain continuously asserted.

While the acceptance of Partial 3, the external computation engine shall assert `segment_step`, indicating that processing shall advance to the next logical processing segment.

The same sequence is then repeated for Segment 1, followed by Segment 2, until every logical processing segment belonging to the current Processing Range has completed reception of its four Partial Transactions.

6.2 Transition to the Accumulation Processing Mode

Upon completion of the final Partial Transaction of the final logical processing segment while operating in `initial_partials` mode, the external computation engine shall assert `phase_change`.

The assertion of `phase_change` terminates `initial_partials` mode and initiates `accumulated_partials` mode without changing the current Processing Range.

Processing resumes from the last logical processing segment that was previously completed.

6.3 Reverse Traversal During Accumulation

While operating in `accumulated_partials` mode, logical processing segments are traversed in the reverse direction: processing begins with the final logical processing segment of the current Processing Range and continues toward the first.

Within every logical processing segment, Partial Transactions continue to be accepted sequentially beginning with Partial 0 and ending with Partial 3.

The `partial_valid` signal shall remain continuously asserted throughout the entire traversal.

While the acceptance of Partial 3 within the current logical processing segment, the external computation engine shall assert `segment_step`, indicating that processing shall continue with the adjacent logical processing segment according to the current traversal direction.

6.4 Traversal Direction Reversal

Whenever processing reaches either boundary of the current Processing Range while operating in `accumulated_partials` mode, the external computation engine shall assert `phase_change`.

Unlike the first assertion of `phase_change`, subsequent assertions do not change the processing mode; instead, each assertion reverses the traversal direction while remaining in `accumulated_partials` mode.

Processing therefore alternates between forward and reverse traversal across the logical processing segments until completion of the current Processing Range.

6.5 Processing Range Completion

Whenever the designated completion point of the current Processing Range is reached, the external computation engine shall assert `next_range`.

The assertion of `next_range` indicates that the current Processing Range has been completed. Immediately thereafter, processing begins for the next Processing Range.

Every newly initiated Processing Range shall always begin from Segment 0 while operating in `initial_partials` mode.

The complete execution sequence described in this chapter is then repeated for every subsequent Processing Range. Processing continues in this manner until the external computation engine asserts `operation_done`, indicating completion of the overall processing operation.

6.6 Worked Example

This example illustrates one complete Processing Range using the default parameter values (`PARTIAL_SIZE` = 4, `OUTPUT_SIZE` = 8, so `PARTIALS_PER_OUTPUT` = 2 and `OUTPUTS_PER_SEGMENT` = 2, per Section 3.1), matching the three-segment range shown in Figure 1.

P0, P1, P2 ... denote successively accepted Partial Transactions. "addr N" denotes the finalized Output Word assigned logical address N (Section 5.4.2), following the sequence Segment 0 → Segment 1 → Segment 2 in `initial_partials` mode.

Cyc.	Control signals asserted	Partial accepted	Resulting action
1	<code>partial_valid</code>	P0	Segment 0, mode = <code>initial_partials</code> .
2	<code>partial_valid</code>	P1	P0+P1 concatenated → stored as addr 0 (initial value).
3	<code>partial_valid</code>	P2	—
4	<code>partial_valid</code> , <code>segment_step</code>	P3	P2+P3 → stored as addr 1. End of Segment 0.
5–8	<code>partial_valid</code> , <code>segment_step</code> on (cyc. 8 only)	P4–P7	Segment 1: addr 2, addr 3 stored (initial values).
9–12	<code>partial_valid</code> , <code>segment_step</code> + <code>phase_change</code> (cyc. 12 only)	P8–P11	Segment 2: addr 4, addr 5 stored. End of <code>initial_partials</code> mode switches to <code>accumulated_partials</code> , traversal starts reverse from Segment 2.
13–16	<code>partial_valid</code> <code>segment_step</code> (cyc. 16)	P12–P15	Segment 2 (accumulate): addr 4, addr 5 updated new values added element-wise to the stored values.
...	...	...	Reverse traversal continues through Segment 1, then Segment 0, accumulating addr 3, addr 2, addr 1, addr 0.

...	partial_valid, ,segment_step phase_change (direction reversal)	—	Range boundary reached traversal direction reverses to forward; accumulation continues Segment 0 → 1 → 2 (per Sections 5.3.3 / 6.4).
...	partial_valid, ,segment_step phase_change, next_range	last partial of range	Range 0 finalized (addr 0–5 final). Range 1 begins at Segment 0, initial_partials.
...	partial_valid, ,segment_step phase_change, next_range,	final partial of operation	No further Partial Transactions accepted for this operation. Module continues
Cyc.	Control signals asserted	Partial accepted	Resulting action
	operation_done		internal processing until all remaining Output Words are finalized.

Independently of this timeline, the downstream interface may present any output_addr value at any time; output_valid and output_data always reflect whether the Output Word at the currently requested address has been finalized yet (Section 5.4), regardless of how far the input stream above has progressed.

7. Buffer Sizing and Optional Timing Requirements

This chapter defines the required buffering capability of the module together with two optional performance targets. These optional targets are intended solely for bonus evaluation and are not mandatory functional requirements. The internal architecture used to satisfy these requirements remains entirely implementation-dependent.

7.1 Maximum Processing Range Size

A single Processing Range shall never contain more than **MAX_SEGMENTS_PER_RANGE = 100** logical processing segments.

This is a fixed upper bound determined by the external computation engine. The module is not required to support Processing Ranges larger than this limit.

7.2 Minimum Buffer Capacity

Because a Processing Range may contain up to **MAX_SEGMENTS_PER_RANGE** logical processing segments, and each logical processing segment produces **OUTPUTS_PER_SEGMENT** Output Words (Section 3.1), the module shall provide sufficient storage capacity to retain at least

$$\text{BUFFER_CAPACITY}(\text{min}) = \text{MAX_SEGMENTS_PER_RANGE} \times \text{OUTPUTS_PER_SEGMENT}$$

simultaneously retrievable Output Words.

With the default parameter values,

$$\text{BUFFER_CAPACITY}(\text{min}) = 100 \times 2 = 200 \text{ Output Words.}$$

ADDR_WIDTH shall always be sufficient to address every retained Output Word. With the default parameter values, **ADDR_WIDTH = 8** provides 256 addressable locations and therefore satisfies this requirement.

7.3 Output Retention Window (Optional Bonus)

The downstream interface is not required to retrieve finalized Output Words immediately after they become available.

Implementations targeting this optional bonus shall guarantee that every finalized Output Word remains retrievable for a downstream retrieval delay of up to

$$\text{MAX_DOWNSTREAM_DELAY} = 3 \times \text{T_RANGE_FILL}(\text{max})$$

where

$$\text{T_SEGMENT} = \text{PARTIAL_SIZE} \text{ clock cycles}$$

is the time required to receive one complete logical processing segment under continuous streaming, and

$$\text{T_RANGE_FILL}(\text{max}) = \text{MAX_SEGMENTS_PER_RANGE} \times \text{T_SEGMENT}$$

is the time required to receive the largest supported Processing Range once.

With the default parameter values,

- **T_SEGMENT = 4 clock cycles**
- **T_RANGE_FILL(max) = 400 clock cycles**
- **MAX_DOWNSTREAM_DELAY = 1200 clock cycles**

This optional requirement allows the downstream interface to postpone reading finalized Output Words without loss of data. Implementations remain free to choose any buffering architecture capable of satisfying this retention window.

7.4 Maximum Completion Latency After operation_done (Optional Bonus)

The assertion of **operation_done** indicates that the final Partial Transaction of the current processing operation has been accepted.

Implementations targeting this optional bonus shall guarantee that every Output Word belonging to the completed processing operation becomes finalized and retrievable no later than **T_SEGMENT** clock cycles after the acceptance of the Partial Transaction associated with **operation_done**.

With the default parameter values, this corresponds to a maximum completion latency of **4 clock cycles**.

This requirement applies only to the completion of the current processing operation after the final Partial Transaction has been accepted and is evaluated independently of the Output Retention Window defined in Section 7.3.